Hazard Analysis
Project Proxi

Team #23
Savinay Chhabra
Amanbeer Singh Minhas
Gourob Podder
Ajay Singh Grewal

April 6, 2026

Table 1: Revision History

Date	Ver.	Notes
10/10/2025	1.0	Added initial hazard analysis.
06/04/2026	1.1	Reformatted revision history to one item per row.
06/04/2026	1.1	Added in-text references to all tables.
06/04/2026	1.1	Replaced ruled tables with Booktabs styling.
06/04/2026	1.1	Rotated the FMEA table for readability.
06/04/2026	1.1	Added external LLM privacy hazard and mitigations.
06/04/2026	1.1	Aligned hazard requirement IDs with the SRS and V&V terminology.

Contents

1 Introduction **1**

2 Scope and Purpose of Hazard Analysis **1**

3 System Boundaries and Components **2**

3.1 System Boundaries 2

3.2 Major System Components 2

4 Critical Assumptions **3**

5 Failure Mode and Effects Analysis **3**

6 Derived Safety and Security Requirements **6**

6.1 Safety Requirements 6

6.2 Security and Privacy Requirements 7

7 Roadmap **8**

Appendix - Reflection Questions **9**

8 Reflections **9**

8.1 Amanbeer Minhas Reflection 9

8.2 Gourob Podder Reflection 10

8.3 Savinay Chhabra Reflection 10

8.4 Ajay Singh Grewal Reflection 11

List of Tables

1 Revision History i

2 Failure Modes and Effects Analysis (FMEA) 4

3 Highest-Priority Mitigations for the Capstone Scope 8

4 Deferred or Longer-Term Mitigations 8

List of Figures

1 Introduction

In Project Proxi, a hazard is any system condition that can lead to harm, loss, or an unacceptable outcome during normal use, foreseeable misuse, or failure. For this project, the most relevant harms are not physical injury, but unintended file changes, privacy loss, accessibility failure, incorrect automation, and loss of user trust.

The purpose of this document is to identify important hazards early and to connect them to mitigations and requirements that remain consistent with the Software Requirements Specification (SRS), Module Guide (MG), Module Interface Specification (MIS), and Verification and Validation (V&V) artifacts. Table 1 summarizes the evolution of this revision.

The rest of the document is organized as follows. Section 2 defines the scope and purpose of the analysis. Section 3 describes system boundaries and major components. Section 4 records critical assumptions. Section 5 presents our Failure Modes and Effects Analysis (FMEA) in Table 2. Section 6 derives safety and security requirements. Finally, Section 7 prioritizes the mitigation roadmap using Tables 3 and 4.

2 Scope and Purpose of Hazard Analysis

Proxi is an AI-powered desktop assistant that supports voice and text input for accessible, high-level computer interaction. Its baseline scope includes speech input, text input, natural-language interpretation, task planning, controlled execution through MCP tools or desktop automation, and textual or spoken feedback.

This hazard analysis focuses on software and interaction hazards within that scope. It covers failures in speech and text handling, intent interpretation, confirmation logic, automation, logging, privacy handling, external service use, and accessibility support. It also considers hazards introduced by interactions with external applications and services when they are part of a supported Proxi workflow.

The following items are outside the direct scope of this analysis:

- physical hardware failure of the user's computer,
- failures of the operating system itself,
- outages of unrelated third-party applications,
- failures of the wider network infrastructure, and
- specialized medical or life-critical device control.

Even with these exclusions, the consequences of software hazards are still significant. Incorrect actions may delete or alter files, disclose personal information, reduce accessibility for target users, or make the system feel unsafe to use. The goal of this document is therefore to identify credible hazards, estimate their effects, and derive practical controls that fit the actual Proxi architecture.

3 System Boundaries and Components

3.1 System Boundaries

Proxi operates at the level of high-level desktop assistance. Inside the system boundary are the components that accept user input, interpret intent, plan actions, enforce confirmation and permission checks, execute supported actions, and present results.

Within the system boundary, Proxi:

- accepts input through speech and text,
- processes requests using intent interpretation and task planning,
- invokes MCP tools or desktop automation for supported actions,
- stores limited local state such as logs and settings, and
- returns visible or spoken feedback to the user.

Outside the system boundary are the operating system, physical hardware, third-party applications, and cloud services that Proxi may depend on but does not fully control. This means hazards can arise both from internal logic and from interfaces with adjacent systems.

3.2 Major System Components

For hazard analysis purposes, the most important conceptual components are:

1. **User Interface Module:** accepts speech or text input and presents feedback.
2. **Natural Language Understanding Engine:** converts raw input into structured intent.
3. **Task Planning and Execution Manager:** decides how to carry out the requested task.
4. **Tool and System Integration Layer:** invokes MCP tools and desktop automation.
5. **Context and Session Manager:** keeps short-term interaction state.
6. **Data Storage and Logging Component:** stores logs, settings, and support information.
7. **Security and Permissions Module:** enforces confirmation, permission scope, and access policies.
8. **External AI Service Interface:** sends approved task-relevant data to external LLM or speech services when those services are enabled.

The last component is especially important in this revision. Because the SRS allows the use of off-the-shelf LLM services, privacy hazards must consider what information is transmitted externally and whether that transmission is necessary and consented to.

4 Critical Assumptions

This analysis is based on the following assumptions:

A1. Proxi runs on supported desktop systems such as Windows and macOS using standard consumer hardware.
A2. Users have a working microphone and display, and can fall back to text interaction when voice input is unreliable.
A3. Supported MCP tools, automation adapters, and adjacent applications generally behave according to their documented interfaces.
A4. Internet connectivity may be unavailable or unstable, but local workflows still remain possible where designed.
A5. High-risk actions are expected to pass through explicit confirmation logic before execution.
A6. When external LLM or speech services are used, Proxi sends only the minimum task-relevant content needed for the requested operation, not a continuous full-screen stream by default.
A7. Users are informed when a workflow requires external processing of personal or sensitive content.

Assumptions A6 and A7 were made explicit in this revision because privacy risk depends strongly on whether Proxi transmits only selected content or an excessive amount of desktop context.

5 Failure Mode and Effects Analysis

Table 2 presents the main FMEA results. The table is rotated to improve readability and to avoid overly compressed text.

Table 2: Failure Modes and Effects Analysis (FMEA)

Component	Failure Mode	Effect of Failure	S	Cause of Failure	O	D	Recommended Action	SRS Ref.	Hazard ID
Speech-to-Text (STT)	Misinterprets voice commands	Wrong action or no action taken	8	Background noise, accent variation, or poor microphone quality	5	6	Improve STT robustness, provide transcript visibility, and allow correction before execution	POA.R.1	H1-1
Command Interpreter	Fails to understand user intent	Task not completed or incorrect action	7	Ambiguous phrasing or missing context	4	6	Add clarification prompts and stronger context handling	FUNC.R.3	H2-1
Tool / MCP Integration	Wrong tool is triggered	Unintended application or function runs	8	Incorrect tool mapping, stale registry entry, or version mismatch	4	6	Validate tool selection, constrain tool scope, and provide safer fallbacks	SAF.R.2, IAS.R.1	H3-1
Desktop Automation	Command crashes, freezes, or affects the wrong target	User task fails or unrelated application state changes	7	Permission errors, fragile selectors, or unresponsive applications	5	5	Add timeouts, retry logic, targeted application checks, and graceful failure handling	ROFT.R.2, WER.R.1	H4-1
Confirmation Layer	High-risk action executes without approval	Accidental deletion, overwrite, or external action	9	Logic bug, missing risk tag, or skipped validation path	3	7	Require explicit confirmation for privileged or destructive actions	SAF.R.1, FUNC.R.9	H5-1
Accessibility Layer	Captions, screen reader flow, or other assistive support fails	Users with disabilities cannot complete supported tasks	9	Missing WCAG support, poor semantics, or inaccessible feedback timing	4	6	Test with assistive technologies and validate against WCAG 2.2 and AODA expectations	ACC.R.2	H6-1
Feedback Manager	Response is delayed, missing, or unclear	User confusion, repeated commands, or loss of trust	5	High load, TTS failure, or poor status messaging	5	5	Provide visible status indicators and clear, plain-language responses	SAL.R.1, UAP.R.1	H7-1

Continued on next page

Table 2 continued from previous page

Component	Failure Mode	Effect of Failure	S	Cause of Failure	O	D	Recommended Action	SRS Ref.	Hazard ID
Update and Deployment	New release breaks a working feature	Supported commands stop working or become inconsistent	6	Incomplete regression testing or packaging errors	4	5	Use regression testing, rollback support, and versioned release artifacts	LON.R.2, REL.R.1	H8-1
Logging and Storage	Logs are missing, corrupted, or over-collected	Traceability is lost or unnecessary user data is retained	6	Logging disabled, storage failure, or excessive capture configuration	4	5	Keep audit logs reliable, bounded, and privacy-aware	AUD.R.1, PRIV.R.2	H9-1
Cross-platform Compatibility	Behaviour differs across operating systems	Features fail on some supported platforms	7	API differences, permission models, or UI variation across OSes	4	5	Test on supported Windows and macOS targets and isolate platform-specific adapters	EPE.R.1, ADP.R.1	H10-1
External AI Service Use	More information than necessary is sent to an external LLM or speech service	Privacy loss, policy non-compliance, or user distrust	9	Poor prompt construction, over-broad context capture, or missing consent	3	7	Minimize outbound data, require user consent, and send only the minimum task-relevant text, file excerpt, or structured fields needed for the request	PRIV.R.2, IAS.R.2	H11-1
External AI Service Use	Sensitive content is sent from the wrong source, such as an unintended window or document	Disclosure of confidential data unrelated to the task	9	Incorrect context selection, stale window focus, or unsafe automation binding	2	8	Require explicit target selection for sensitive operations and avoid whole-screen capture by default	PRIV.R.2, SAF.R.2	H11-2

Legend: S = Severity (1–10), O = Occurrence (1–10), D = Detection (1–10).

The highest-risk items in Table 2 are the failure to confirm high-risk actions and the privacy hazards introduced by external AI service use. These hazards received the highest severity because they can cause data loss or disclosure even when the rest of the system appears to be operating normally.

The new privacy rows clarify the TA's concern directly. In the intended Proxi design, the system should not send the user's whole screen by default. Instead, it should send only the minimum information needed for the specific task, such as the current command text, a selected file excerpt, or a small, explicitly chosen content segment.

6 Derived Safety and Security Requirements

The hazards in Table 2 lead to the following requirements. These are phrased to remain consistent with the terminology and intent of the SRS, MG, MIS, and V&V plan.

6.1 Safety Requirements

SAF.R.1 - Explicit confirmation for risky actions

Any privileged, destructive, or externally impactful action shall require explicit user confirmation before execution.

Why: This mitigates H5-1 and reduces the chance of silent harmful execution.

Verification idea: Test cases for deletes, overwrites, sends, and removals should show a confirmation step every time.

SAF.R.2 - Scoped and validated execution

Proxi shall validate targets, tool choice, and execution scope before running automation or external integrations.

Why: This mitigates H3-1, H4-1, and H11-2.

Verification idea: Out-of-scope targets, risky destinations, and ambiguous application targets should trigger blocking or clarification.

ROFT.R.2 - Graceful failure handling

Supported workflows shall fail safely when tools, applications, or network dependencies are unavailable.

Why: This mitigates H4-1 and H8-1.

Verification idea: Simulated timeouts and unavailable adapters should end in a clear failure state rather than partial unsafe execution.

ACC.R.2 - Accessible recovery and feedback

Safety prompts, status messages, and recovery paths shall remain accessible to users who rely on voice, captions, scaling, or assistive technologies.

Why: A safety control that cannot be perceived or understood does not adequately control risk for the target users.

Verification idea: Confirmation and error flows should be checked with screen readers, captions, and enlarged text.

6.2 Security and Privacy Requirements

PRIV.R.2 - Data minimization for external services

When Proxi uses an external LLM, STT, or TTS service, it shall send only the minimum task-relevant data needed to complete the request.

Why: This directly mitigates H11-1 and aligns with the SRS rule that only strictly needed data should leave the user's computer.

Verification idea: Inspect representative outbound requests and confirm that they contain only the selected text, command, or excerpt rather than broad desktop context.

PRIV.R.3 - Explicit consent for external sharing

Proxi shall clearly inform the user when a workflow requires data to be processed by an external AI service and shall obtain consent before sending personal or sensitive content.

Why: This mitigates H11-1 and supports informed user control.

Verification idea: Privacy-sensitive workflows should present a consent step before transmission.

ACS.R.2 - Permission-bounded access

The system shall operate only within the permission scope explicitly granted by the user for folders, applications, and external actions.

Why: This mitigates H3-1, H4-1, and H11-2.

Verification idea: Requests outside allowed scope should be blocked or require a new permission grant.

AUD.R.1 - Traceable and privacy-aware audit logging

The system shall keep audit logs for actions, confirmations, and outcomes without silently storing unnecessary personal content.

Why: This mitigates H9-1 while preserving debugging and accountability.

Verification idea: Logs should record action metadata and outcome state, but should avoid excessive raw content retention.

IAS.R.2 - Protected communication with adjacent systems

External integrations shall use supported protected communication mechanisms, such as HTTPS, when network transport is required.

Why: This reduces exposure to interception and tampering during external service use.

Verification idea: Network inspection should confirm encrypted transport for supported cloud-backed workflows.

7 Roadmap

Table 3 prioritizes the mitigation work that most directly reduces risk in the capstone scope. Table 4 lists useful follow-on work that is valuable but less critical for the final milestone.

Table 3: Highest-Priority Mitigations for the Capstone Scope

ID	Mitigation to Implement	Target Window
SAF.R.1	Confirmation step for destructive, privileged, or externally impactful actions	Sprint 1
SAF.R.2	Stronger target validation and scoped execution checks	Sprint 1
PRIV.R.2	Data minimization for outbound LLM and speech-service requests	Sprint 1–2
PRIV.R.3	Consent flow for privacy-sensitive external processing	Sprint 2
AUD.R.1	Reliable and bounded audit logging for actions and outcomes	Sprint 2
ROFT.R.2	Safe timeout, retry, and failure handling in adapters	Sprint 2

Table 4: Deferred or Longer-Term Mitigations

ID	Mitigation	Reason for Deferral
ACS.R.2	Deeper sandboxing and finer-grained permission profiles	Requires more platform-specific engineering than fits the capstone timeline
IAS.R.2	Broader automated inspection of all external service traffic	Best added after the baseline supported workflows are stabilized
REL.R.1	More comprehensive release hardening and signing workflow	Useful for long-term deployment, but not strictly required for the course release

Appendix - Reflection Questions

The purpose of these reflection questions is to help the team assess what it learned while creating the hazard analysis and how the process could improve in future projects.

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. Which risks had your team already considered before this deliverable, and which ones emerged during the hazard analysis itself?
4. Other than physical harm, what other categories of software risk are important to consider, and why?

8 Reflections

8.1 Amanbeer Minhas Reflection

1. What went well while writing this deliverable?
   Writing this deliverable went well because we already had a clear idea of our system from the SRS and FMEA work. Breaking the project into components helped me think about hazards more systematically and kept the analysis organized. I also felt more confident in identifying potential risks and writing about them in a structured way.

2. What pain points did you experience during this deliverable, and how did you resolve them?
   One challenge was figuring out how detailed to make each section without overcomplicating the document. I also found it tricky to write about hazards without repeating the same points as the SRS. I resolved this by focusing on specific examples from our project, like MCP integration and OS automation, and connecting them directly to potential risks.

3. Which of your listed risks had your team thought of before this deliverable, and which did you think of while doing this deliverable?
   We had already thought about risks like incorrect command execution and permission issues earlier in the project. However, risks such as operating system compatibility problems and deployment-related failures came up while writing this hazard analysis. They became more obvious once we broke the system into components and looked deeper into how each part could fail.

4. Other than the risk of physical harm, list at least two other types of risk in software products. Why are they important to consider?

One major risk is data loss or corruption, which can harm user trust and make the system unreliable. Another risk is accessibility failure, where users with limitations might not be able to use the software as intended. Both are important because they affect user confidence, usability, and the overall success of the product.

8.2 Gourob Podder Reflection

1. What went well while writing this deliverable?
   Identifying system boundaries and major components went smoothly because the project architecture was already clear from prior SRS work. Defining modules like the User Interface, Natural Language Understanding, and Agent Manager helped organize potential hazards systematically. Team collaboration also allowed us to quickly align on how risks could manifest in both software and user interactions.

2. What pain points did you experience during this deliverable, and how did you resolve them?
   Determining the appropriate level of detail for the hazard analysis without drifting into system design was challenging. It was sometimes difficult to distinguish between a hazard and a general usability concern. We resolved this by using safety engineering definitions and focusing on sources of harm or adverse outcomes rather than design flaws. Abstract risks such as privacy or misinterpretation of user intent were addressed using scenario-based brainstorming.

3. Which of your listed risks had your team thought of before this deliverable, and which did you think of while doing this deliverable?
   We had already recognized usability and privacy risks, like misinterpreted commands or unauthorized data access. New risks emerged during the hazard analysis, especially related to component interactions, such as potential failures in the Task Planning module or dependency on unreliable external APIs. These became apparent as we examined each subsystem and its interface boundaries.

4. Other than the risk of physical harm, list at least two other types of risk in software products. Why are they important to consider?
   Privacy and data security risks, such as improper handling of sensitive user data, can harm user trust and violate legal requirements. Reliability and availability risks, like system downtime or malfunctions, can prevent users from completing essential tasks, especially for assistive technologies. Both are critical for maintaining usability, accessibility, and overall user confidence.

8.3 Savinay Chhabra Reflection

1. What went well while writing this deliverable?
   The provided template helped structure our hazards. Collaboration and work distribu-

tion went more smoothly, and we could relate hazards directly to the well-structured SRS document.

2. What pain points did you experience during this deliverable, and how did you resolve them?
   Staying solution-neutral while analyzing hazards was difficult. The line between requirements and design is blurred, and we sometimes discussed solutions prematurely. We resolved this by refocusing on hazards rather than proposed solutions.

3. Which of your listed risks had your team thought of before this deliverable, and which did you think of while doing this deliverable?
   Initially, we focused on privacy and legal risks associated with data breaches. While performing hazard analysis, we discovered additional potential risks and mitigation strategies that were not previously considered.

4. Other than the risk of physical harm, list at least two other types of risk in software products. Why are they important to consider?
   Legal risks due to data breaches or violations of user privacy, and compliance risks from accessibility failures in design, are both important because they affect user trust, legal standing, and the usability of the product.

8.4 Ajay Singh Grewal Reflection

1. What went well while writing this deliverable?
   Turning the hazards into short, testable requirements worked well. By adding criteria under each requirement, it became easier to understand how to verify them later. The FMEA table also helped organize the hazards clearly, and breaking the system into components made it easier to think about specific risks.

2. What pain points did you experience during this deliverable, and how did you resolve them?
   Choosing criteria that were realistically achievable within our time frame was tricky. We had to balance being thorough with being practical. We resolved this by focusing on the highest-impact hazards that were most relevant to our project scope and user needs.

3. Which of your listed risks had your team thought of before this deliverable, and which did you think of while doing this deliverable?
   We had already discussed risks like confirmation for destructive actions and permission issues. While writing the hazard analysis, we identified additional risks like OS compatibility and deployment-related failures.

4. Other than the risk of physical harm, list at least two other types of risk in software products. Why are they important to consider?
   One important risk is sending more data than needed, or sending it without user consent, because that breaks user trust and may discourage continued use. Another

important risk is reliability failure, where a bad update or broken tool makes the system unusable. Both matter because they affect user trust, usability, and long-term adoption.